Enabling Auto-Correction on Soft Braille Keyboard

DAN ZHANG, Computer Science Department, Stony Brook University, USA
YAN MA, Computer Science Department, Stony Brook University, USA
GLENN DAUSCH, Computer Science Department, Stony Brook University, USA
WILLIAM H SEIPLE, Lighthouse Guild, USA
DAVID XIANFENG GU, Department of Computer Science, Stony Brook University, USA
IV RAMAKRISHNAN, Computer Science Department, Stony Brook University, USA
XIAOJUN BI, Department of Computer Science, Stony Brook University, USA

A soft Braille keyboard is a graphical representation of the Braille writing system on smartphones. It provides an essential text input method for visually impaired individuals, but accuracy and efficiency remain significant challenges. We present an intelligent Braille keyboard with auto-correction ability, which uses optimal transportation theory to estimate the distances between touch input and Braille patterns, and combines it with a language model to estimate the probability of entering words. The proposed system was evaluated through both simulations and user studies. In a touch interaction simulation on an Android phone and an iPhone, our intelligent Braille keyboard demonstrated superior error correction performance compared to the Android Braille keyboard with proofreading suggestions and the iPhone Braille keyboard with spelling suggestions. It reduced the error rate from 55.81% on Android and 57.13% on iPhone to 19.80% under high typing noise. Furthermore, in a user study of 12 participants who are legally blind, the intelligent Braille keyboard reduced word error rate (WER) by 59.5% (42.53% to 17.28%) with a slight drop of 0.74 words per minute (WPM), compared to a conventional Braille keyboard without auto-correction. These findings suggest that our approach has the potential to greatly improve the typing experience for Braille users on touchscreen devices.

ACM Reference Format:
Dan Zhang, Yan Ma, Glenn Dausch, William H Seiple, David Xianfeng Gu, IV Ramakrishnan, and Xiaojun Bi. 2026. Enabling Auto-Correction on Soft Braille Keyboard. 1, 1 (January 2026), 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Smartphones have become indispensable tools for modern life, yet they present significant accessibility barriers for people who are blind or visually impaired. Text entry, a fundamental interaction method for activities ranging from messaging to web browsing, remains particularly challenging. Standard QWERTY soft keyboards are inefficient for blind users, often yielding low typing speeds of around 5 words per minute (WPM) and high error rates [3, 12, 15]. While voice input offers speed, it poses privacy concerns and makes error correction difficult [8].

Authors' Contact Information: Dan Zhang, Computer Science Department, Stony Brook University, Stony Brook, New York, USA, zhang64@cs.stonybrook.edu; Yan Ma, Computer Science Department, Stony Brook University, Stony Brook, New York, USA, yanma1@cs.stonybrook.edu; Glenn Dausch, Computer Science Department, Stony Brook University, Stony Brook, New York, USA, glenn.dausch@stonybrook.edu; William H Seiple, Lighthouse Guild, New York, New York, USA, wseiple@lighthouseguild.org; David Xianfeng Gu, Department of Computer Science, Stony Brook University, Stony Brook, New York, USA, gu@cs.stonybrook.edu; IV Ramakrishnan, Computer Science Department, Stony Brook University, Stony Brook, New York, USA, ram@cs.stonybrook.edu; Xiaojun Bi, Department of Computer Science, Stony Brook University, Stony Brook, New York, USA, xiaojun@cs.stonybrook.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

Soft Braille keyboards, which replicate the six-dot Braille layout on touchscreens, offer a promising alternative. However, current implementations on major mobile operating systems deterministically map touch inputs to Braille dots. This lack of intelligence means that even slight deviations in finger placement result in errors. Research has shown that error rates on these keyboards can exceed 40% for some user groups [6].

While auto-correction via statistical decoding has revolutionized QWERTY typing for sighted users [7], these techniques are not directly transferable to Braille. Standard decoders assume a single touch point per character mapped via a Gaussian distribution [2]. In contrast, Braille characters are formed by simultaneous multi-touch patterns (chords) where the number of touch points may not even match the target character's dot count due to input errors.

To address this gap, we propose an intelligent Braille keyboard powered by Optimal Transport (OT) theory [17]. This mathematical framework calculates the minimal cost to transform the user's touch distribution into the standard Braille dot pattern distribution. By integrating this spatial model with a language model, our system can infer the intended word even amidst significant input noise.

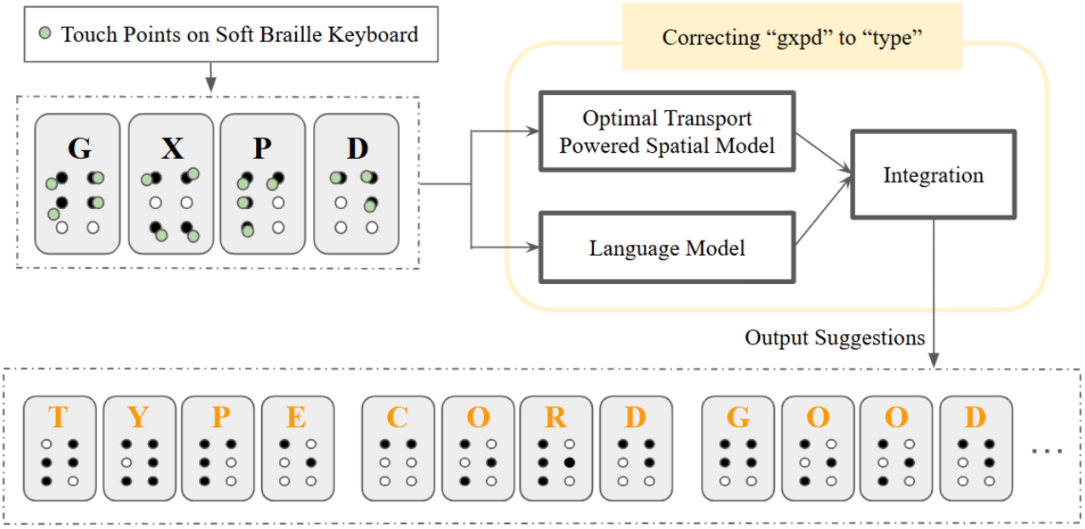


Fig. 1. Illustration of word auto-correction on the proposed intelligent Braille keyboard. In this example, the user wants to type word "type", but mistakenly has a dispositional touch for letter 't', an omitted touch for letter 'y', and an extra touch for letter 'e'. The touch input positions are initially mapped to letters: 'g', 'x', 'p', and 'd'. The keyboard then utilizes two modules—spacial model powered by optimal transport and a language model—to compute the likelihood of each candidate word being the intended word. After integrating these probabilities, the keyboard displays the top three candidate words with the highest probabilities at the bottom: "type", "cord", and "good".

2 Background and Related Work

2.1 Text Entry Techniques for Visually Impaired Individuals

The primary method for text entry among blind users is the "explore by touch" method on QWERTY keyboards, which is notoriously slow. Studies indicate average speeds of approximately 5 WPM, drastically slower than the 36 WPM

average for sighted users [14]. While voice input is an alternative, correcting recognition errors is a tedious process requiring precise cursor manipulation, often consuming up to 80% of the interaction time [1].

Soft Braille keyboards utilize the screen to represent the Perkins Brailler layout. While they theoretically allow for faster entry, the lack of tactile feedback on glass surfaces leads to high error rates due to finger drift and imprecise targeting [13].

2.2 Statistical Decoding

In the domain of virtual keyboards, statistical decoding is the standard for error correction. It treats touch points as signals distributed spatially. Using Bayes' rule, the probability of a word is calculated by combining the spatial probability of the touch points with the prior probability from a language model [7]. While advanced models like the Dual Gaussian distribution [2] handle finger variability well, they are designed for single-finger tapping sequences, not the multi-touch chording inherent to Braille.

2.3 On-screen Braille Keyboards

Various designs have been explored, such as BrailleTouch [6] and TypeInBraille [9]. Commercial implementations exist in iOS and Android (TalkBack). However, error correction in these systems is generally limited to basic spell-checking after the fact, rather than correcting the touch input itself. Exceptions like B# [11] use chord-based correction but do not utilize the robust spatial mapping offered by Optimal Transport.

3 Method: Intelligent Braille Keyboard

Our system improves upon standard Braille keyboards by introducing a decoding algorithm that handles the ambiguity of touch inputs.

3.1 Decoding Principle

The decoding process parallels QWERTY statistical decoders but employs a novel spatial model. We compute the posterior probability of a word candidates using two components:

(1) A Spatial Model: Infers the probability of a character based on touch locations.
(2) A Language Model: Infers the probability based on linguistic context.

3.2 Representing Inputs as Distributions

The core innovation is representing both the user's input and the target Braille characters as discrete mass distributions.

- **User Input (p):** If a user touches the screen with m fingers, we treat this as a distribution of mass 1, evenly divided among the m points ($1/m$ mass at each location).
- **Braille Pattern (c):** A Braille character with n dots is represented as mass 1 evenly distributed among the n standard dot locations ($1/n$ mass each).

This formulation allows us to compare inputs where the number of touches does not match the number of required dots (e.g., touching 4 points for a 2-dot letter).

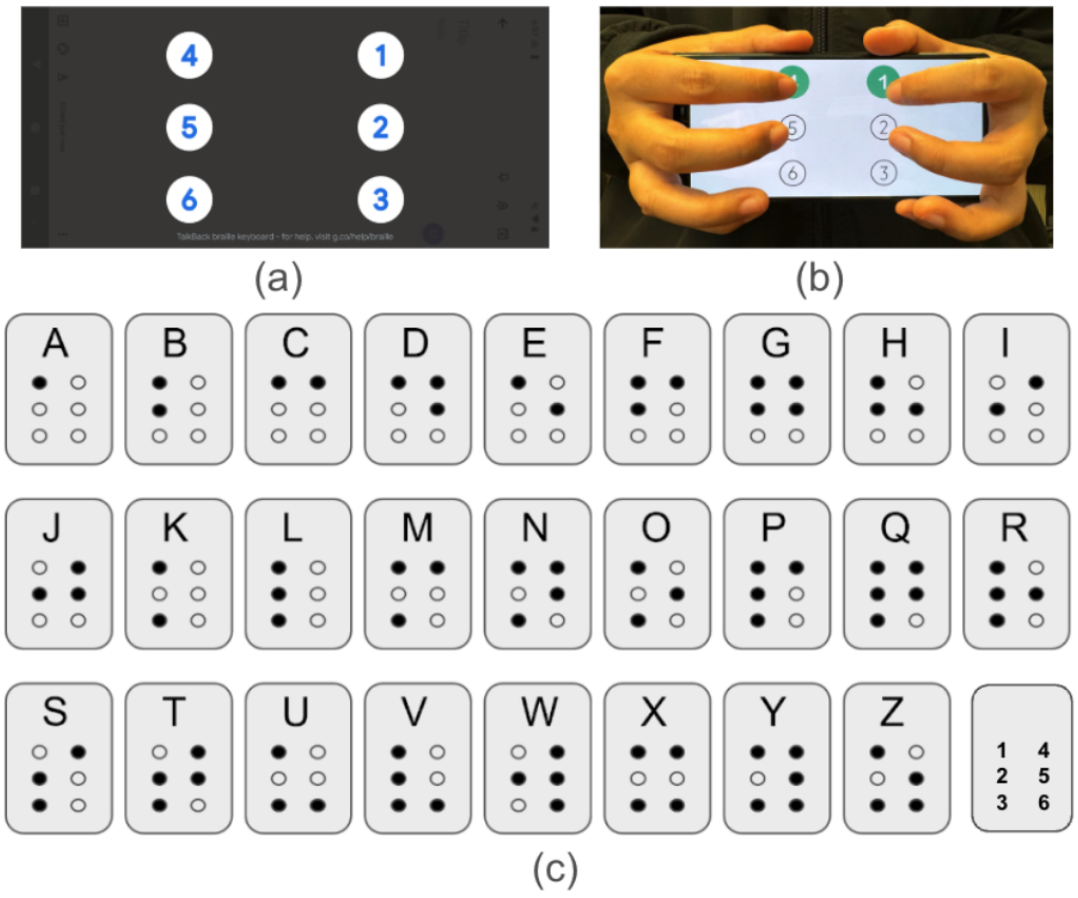


Fig. 2. (a) Screenshot of an existing conventional Braille keyboard. (b) Entering 'c' on the intelligent Braille keyboard. (c) Mappings between Braille input and letters, and the numbering of Braille dots. Note that the numbering of Braille dots on the touchscreen mirrors the layout in (c), as the touchscreen is facing away during typing.

3.3 Squared Wasserstein-2 Distance

To quantify the difference between the user's input p and a target character c, we utilize the Optimal Transport theory, specifically the squared Wasserstein-2 distance (W_2^2). This metric effectively measures the "effort" to move the mass from the touch points to the Braille dot locations.

First, a cost matrix M is generated, where M_{ij} is the squared Euclidean distance between a touch point p_i and a Braille dot location q_j:

$$M_{ij} = ||p_i - q_j||^2 \tag{1}$$

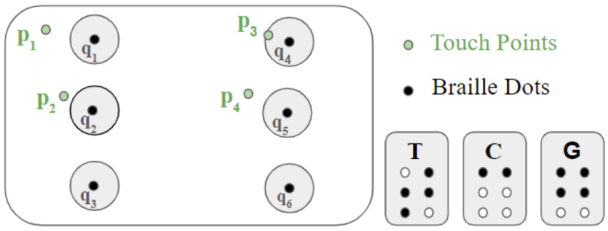


Fig. 3. Illustration of Braille input and the Braille dot patterns of three candidate characters, 't', 'c' and 'g', for the example in Section 3.2 and 3.3.

The distance is then found by minimizing the transportation cost:

$$W_2^2(p, c) = \min_{Y \in T} \sum_{i=1}^{m} \sum_{j=1}^{6} M_{ij} Y_{ij} \tag{2}$$

where Y represents the transportation plan. We solve this using the network simplex method via the POT package [5].

3.4 Spatial and Language Scoring

The calculated transport cost is converted into a spatial probability using a Gaussian function:

$$P(c|p) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{W_2^2(p, c)}{2\sigma^2}\right) \tag{3}$$

Finally, this spatial score is combined with a bigram language model score $L(w)$ (trained on the COCA corpus) to determine the final candidate score.

4 Comparison with Android and iPhone Keyboards

To validate our approach, we conducted extensive simulations comparing our Intelligent Braille Keyboard against the native correction features of Android (TalkBack proofread) and iOS (VoiceOver spellcheck).

4.1 Simulation Setup

We utilized automation tools (AutoTouch for iOS, Appium for Android) to simulate typing on physical devices (iPhone 6s and Google Pixel 6). The test set consisted of 200 sentences from a standard text entry dataset [16].

4.2 Simulation I: Statistical Noise

We introduced Gaussian noise to the touch coordinates to simulate imprecise typing. We tested at two noise levels: High ($SD = 7.8$ mm) and Low ($SD = 3.9$ mm). As shown in Table 1, our system significantly outperformed both competitors. Under high noise, our Top-1 Error Rate was only 19.80%, compared to over 55% for both Android and iPhone.

4.3 Simulation II: Specific Typing Errors

We also simulated specific error types common in Braille typing:

- **Disposition:** Touching the wrong dots.

Table 1. Simulation I results: WER (%) with Gaussian noise on Android Braille keyboard, iPhone Braille keyboard, and our intelligent Braille keyboard with auto-correction

Braille Keyboards	SD = 7.8 mm		SD = 3.9 mm	
	Top-1	Top-3	Top-1	Top-3
Android	55.81	52.74	7.49	6.82
iPhone	57.13	52.82	8.63	7.96
Auto-correction	19.80	12.29	1.30	0.00

- **Omission:** Missing a required dot.
- **Extra:** Adding an unintended dot.

Table 2 highlights the robustness of our OT-based decoder. While existing keyboards struggled significantly (WER > 45%), our system maintained low error rates, particularly for omitted and extra dots.

Table 2. Simulation II results: WER (%) for handling different typing errors on Android Braille keyboard, iPhone Braille keyboard, and our intelligent Braille keyboard

Braille Keyboards	Disposition Top-1/Top-3	Omitted Top-1/Top-3	Extra Top-1/Top-3
Android	64.79 / 60.02	64.41 / 60.50	59.06 / 54.58
iPhone	55.29 / 46.46	48.58 / 37.85	47.36 / 34.39
Auto-correction	19.54 / 10.49	11.49 / 6.90	14.02 / 8.04

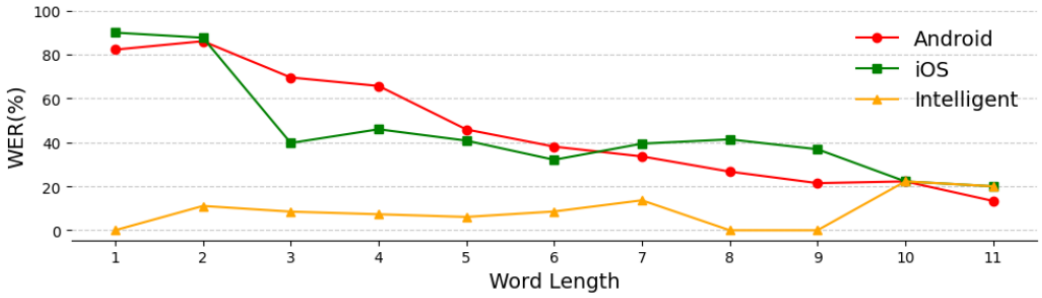


Fig. 4. WER on Top-3 Suggestions Across Different Word Lengths

4.4 Simulation III: Multi-letter Errors

We further tested the system's ability to correct words containing errors in two separate letters. As shown in Table 3, the system remained effective, though disposition errors remained the most challenging to correct.

Table 3. Simulation III results: WER (%) results with two letters mistyped in a word on our Intelligent Braille keyboard

	Disposition	Omitted	Extra
Top-1	34.79	19.08	17.93
Top-3	20.96	8.59	5.06

5 Findings and Results: User Evaluation

To assess real-world performance, we conducted a user study with 12 legally blind participants (6 men, 6 women, average age 49.3).

5.1 Apparatus and Procedure

We implemented the Intelligent Braille Keyboard as a web-based prototype running on a Google Pixel 6. Participants held the device in "screen-away" mode (fingers curling back to touch the screen), as depicted in Figure 5. The task involved transcribing phrases selected from the T-40 dataset.



Fig. 5. A user is typing letter "g" in screen-away mode on the intelligent Braille keyboard

5.2 Input Speed and Error Rate

The study compared the intelligent keyboard against the standard TalkBack Braille keyboard.

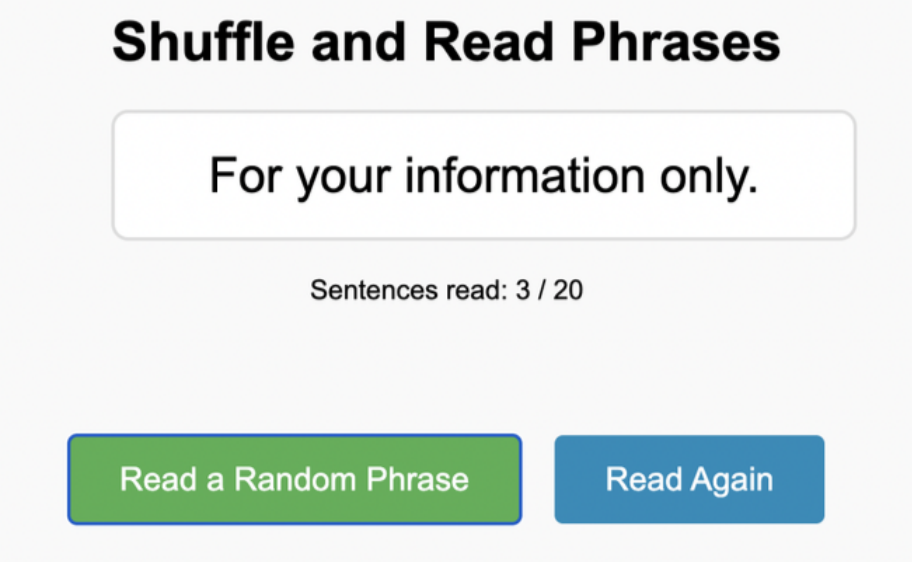


Fig. 6. A webpage screenshot of our transcription task

- **Word Error Rate (WER):** The intelligent keyboard dramatically reduced errors. The average WER dropped from 42.53% (Conventional) to 17.28% (Intelligent). This represents a 59.5% improvement.
- **Typing Speed (WPM):** There was a slight reduction in speed (6.18 WPM Conventional vs. 5.44 WPM Intelligent). This was attributed to the time required for users to navigate the suggestion lists provided by the auto-correction system.

Figure 7 illustrates the individual WER improvements, showing that almost every participant benefited from the auto-correction.

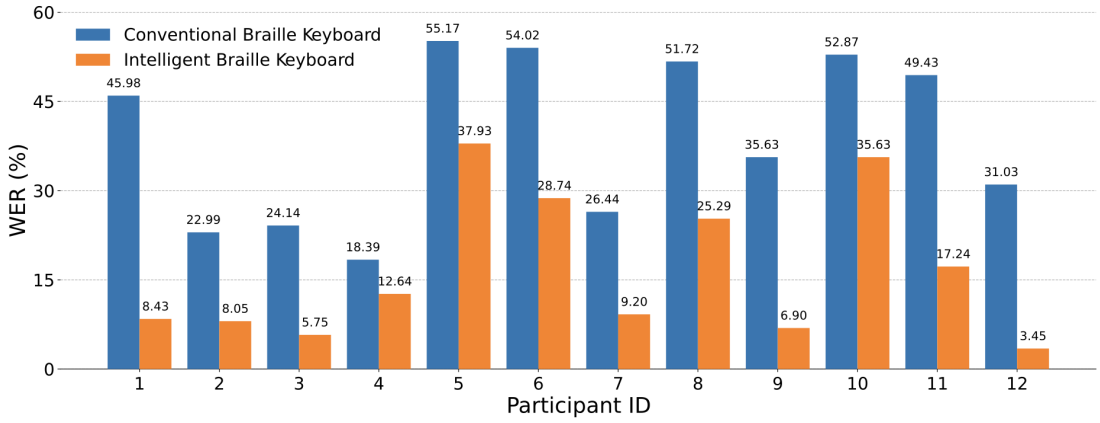


Fig. 7. Average WER for the 12 participants

5.3 Learning Effect

We analyzed performance across four blocks of trials (Figure 8). While the intelligent keyboard initially had higher error rates as users adapted to the system, the accuracy improved significantly by Block 3. By Block 4, users achieved speeds of 6.81 WPM, nearing the conventional keyboard's speed, but with much higher accuracy.

5.4 Suggestion Usage

Table 4 details how participants utilized the suggestion feature (activated by swiping up). The high accuracy rate of accepted suggestions (mean 74.5%) validates the effectiveness of the language and spatial model integration.

Table 4. Analysis of Suggestion Usage: Frequency of Swipe-Up Gestures and Correct Suggestions Adopted

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	Mean
Suggestion Usage	17	14	15	24	36	25	16	30	13	45	25	19	23.3
Correct Suggestion	15	10	12	17	25	15	15	24	9	29	17	15	16.9
Accuracy Rate (%)	88.2	71.4	80.0	70.8	69.4	60.0	93.8	80.0	69.2	64.4	68.0	78.9	74.5

5.5 Usability

Participants rated the systems using the System Usability Scale (SUS) [4]. The Intelligent Braille Keyboard received a significantly higher median score of 81.5 compared to 71.25 for the conventional keyboard, indicating higher user satisfaction despite the slight speed trade-off.

6 Discussion

The results from both the simulation and the user study provide compelling evidence that integrating Optimal Transport-based decoding into Braille keyboards significantly enhances accessibility.

6.1 Accuracy Improvements

The most critical finding is the massive reduction in error rates. For visually impaired users, correcting errors after typing is cognitively demanding and time-consuming. By improving the Top-1 suggestion accuracy (as seen in the simulation results), we minimize the need for manual navigation through suggestion lists. The user study confirmed this, with a near 60% reduction in WER.

6.2 Individual Differences

We observed significant variance among participants. Some users, likely those less proficient in Braille or with less precise motor control, benefited immensely from the correction. Others who were already fast typists saw smaller gains. This suggests that future iterations of the keyboard could benefit from personalization, adapting the "spread" of the spatial model (σ) to the individual user's typing precision.

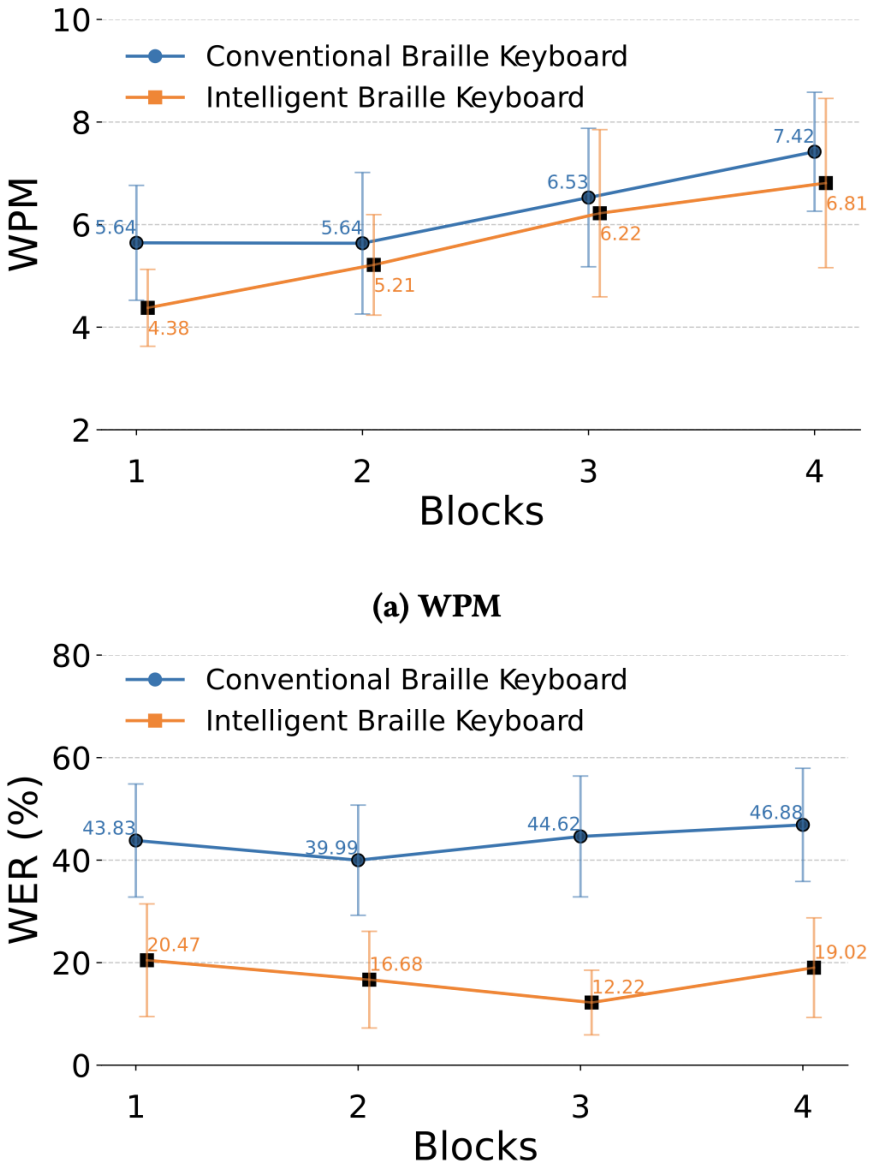


Fig. 8. Average(95%CI) of WPM and WER(%) by block

6.3 Promoting Braille Literacy

Braille literacy has been declining, yet it remains a crucial skill for employment and independence among the blind community [10]. By making Braille text entry on smartphones—the most ubiquitous computing device—less frustrating and more accurate, we hope to encourage more users to practice and utilize Braille daily.

6.4 Limitations and Future Work

The primary limitation observed was the slight decrease in typing speed. This is a common trade-off when introducing suggestion mechanisms. Future work will focus on optimizing the interaction design to make selecting suggestions faster, perhaps through gesture shortcuts. Additionally, we plan to explore the use of Large Language Models (LLMs) to replace the bigram model, potentially improving the contextual prediction capabilities even further.

7 Conclusion

We designed and implemented a novel intelligent Braille keyboard with auto-correction for smartphones that can automatically correct errors while typing in Braille. We created an intelligent decoder that uses optimal transportation theory to calculate the distances between touch input and Braille patterns, and combine it with a language model to estimate the probability of entering a word. We compared the error correction capability of our intelligent Braille keyboard powered by the proposed decoder with that of the existing TalkBack Braille keyboard and the iPhone Braille keyboard by simulating touch interactions on an Android smartphone and an iPhone. The results showed that our intelligent Braille keyboard greatly reduces typing errors compared to the Android's proofread feature and the iPhone's spellchecker for Braille input. It reduced the error rate from 55.81% (Top-1) and 52.74% (Top-3) on Android and 57.13% (Top-1) and 52.82% (Top-3) on iPhone to just 19.80% (Top-1) and 12.29% (Top-3) under high typing noise conditions, and demonstrated very low error rates under lower typing noise. Furthermore, in a user study involving 12 participants, we compared the performance of our keyboard against a conventional Braille keyboard without auto-correction. The results indicated that our keyboard significantly reduced the word error rate, from 42.53% to 17.28%. The Braille keyboard provides a promising method for accurate text entry for blind users on smartphones.

References

[1] Shiri Azenkot and Nicole B. Lee. 2013. Exploring the Use of Speech Input by Blind People on Mobile Devices. In *Proceedings of the 15th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '13)*. ACM, New York, NY, USA, 1–8. doi:10.1145/2513383.2513440

[2] Xiaojun Bi and Shumin Zhai. 2016. Predicting Finger-Touch Accuracy Based on the Dual Gaussian Distribution Model. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (UIST '16)*. ACM, New York, NY, USA, 313–319. doi:10.1145/2984511.2984546

[3] Syed Masum Billah, Yu-Jung Ko, Vikas Ashok, Xiaojun Bi, and IV Ramakrishnan. 2019. Accessible Gesture Typing for Non-visual Text Entry on Smartphones. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. ACM, New York, NY, USA, 1–12.

[4] John Brooke. 1996. SUS: A 'Quick and Dirty' Usability Scale. *Usability Evaluation in Industry* 189, 3 (1996), 189–194.

[5] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, et al. 2021. POT: Python Optimal Transport. *Journal of Machine Learning Research* 22, 78 (2021), 1–8.

[6] Brian Frey, Caleb Southern, and Mario Romero. 2011. BrailleTouch: Mobile Texting for the Visually Impaired. In *Proceedings of the 6th International Conference on Universal Access in Human-Computer Interaction: Context Diversity - Volume Part III (UAHCI '11)*. Springer-Verlag, Berlin, Heidelberg, 19–25.

[7] Joshua Goodman, Gina Venolia, Keith Steury, and Chauncey Parker. 2002. Language Modeling for Soft Keyboards. In *Proceedings of the 7th International Conference on Intelligent User Interfaces (IUI '02)*. ACM, New York, NY, USA, 194–195. doi:10.1145/502716.502753

[8] Jonggi Hong, Christine Vaing, Hernisa Kacorri, and Leah Findlater. 2020. Reviewing Speech Input with Audio: Differences between Blind and Sighted Users. *ACM Transactions on Accessible Computing* 13, 1 (2020), 1–28.

[9] Sergio Mascetti, Cristian Bernareggi, and Matteo Belotti. 2011. *TypeInBraille: Quick Typing on Smart Phones by Blind Users.* Technical Report. Universita degli Studi di Milano, Milan, Italy. Retrieved from http://homes.di.unimi.it/mascetti/Sergio_Mascetti__home_page/Research_files/TR39-2011.pdf.

[10] National Federation of the Blind. 2009. The Braille Literacy Crisis in America: Facing the Truth, Reversing the Trend, Empowering the Blind. *Braille Monitor* 52, 5 (2009).

[11] Hugo Nicolau, Kyle Montague, Tiago Guerreiro, João Guerreiro, and Vicki L. Hanson. 2014. B#: Chord-based Correction for Multitouch Braille Input. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '14)*. ACM, New York, NY, USA, 1705–1708.

[12] Hugo Nicolau, Kyle Montague, Tiago Guerreiro, André Rodrigues, and Vicki L. Hanson. 2015. Typing Performance of Blind Users: An Analysis of Touch Behaviors, Learning Effect, and In-situ Usage. In *Proceedings of the 17th International ACM SIGACCESS Conference on Computers & Accessibility (ASSETS '15)*. ACM, New York, NY, USA, 273–280.

[13] João Oliveira, Tiago Guerreiro, Hugo Nicolau, Joaquim Jorge, and Daniel Gonçalves. 2011. BrailleType: Unleashing Braille over Touch Screen Mobile Phones. In *Proceedings of the 13th IFIP TC 13 International Conference on Human-Computer Interaction - Volume Part I (INTERACT '11)*. Springer-Verlag, Berlin, Heidelberg, 100–107.

[14] Kseniia Palin, Anna Feit, Sunjun Kim, Per Ola Kristensson, and Antti Oulasvirta. 2019. How do People Type on Mobile Devices? Observations from a Study with 37,000 Volunteers. In *Proceedings of 21st International Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI '19)*. ACM, New York, NY, USA. doi:10.1145/3338286.3340114

[15] Caleb Southern, James Clawson, Brian Frey, Gregory Abowd, and Mario Romero. 2012. An evaluation of BrailleTouch: mobile touchscreen text entry for the visually impaired. In *Proceedings of the 14th International Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI '12)*. ACM, New York, NY, USA, 317–326.

[16] Keith Vertanen and Per Ola Kristensson. 2011. A Versatile Dataset for Text Entry Evaluations Based on Genuine Mobile Emails. In *Proceedings of the 13th International Conference on Human Computer Interaction with Mobile Devices and Services (MobileHCI '11)*. ACM, New York, NY, USA, 295–298.

[17] Cédric Villani. 2009. *Optimal Transport: Old and New.* Vol. 338. Springer, Berlin, Heidelberg.